A NUMERICAL METHOD FOR SOLVING THE EIKONAL EQAUATION ON SIMPLICIAL TESSELATIONS*

SACHIN NATESH†

Abstract. In this paper, we describe a method for solving the Eikonal equation on domains in $\mathbb{R}^2$ or $\mathbb{R}^3$ tesselated by triangles or tetrahedra. Our method generalizes to simplices in higher dimensions, owing to our choice of solution representation under the Jacobi family of orthogonal polynomials on the simplex. First, we describe a spectral decomposition for functions supported on the simplex in terms of their weighted L_2 expansion coefficients under the Jacobi basis. Computing this expansion requires an accurate and efficient quadrature, and we provide a novel optimization routine based on approximate joint-diagonalization of Jacobi matrices for generating such quadratures. Then, we consider differential operators acting on coefficients of a Jacobi expansion. Recent results due to Townsend, Oliver and Vasil enable the construction of sparse differential operators on the standard triangle, interoperating within the Jacobi polynomial family parameter space, and leading to low-storage, banded discrete differential operators. We generalize these results to the tetrahedra (laying the path for generalizing to $d > 3$), and use the resulting differential operators to construct the Eikonal operator on the simplex. Ultimately, we must solve a non-linear PDE, and working in coefficient space implies that our solution variables must be thought of as functions. Hence, we reformulate the Eikonal problem in terms of an infinite-dimensional L_2 mininization problem, which can be discretized with the aforementioned numerical quadrature. As with the README, this is more of a blog, but it's much easier to typeset math here.

Key words. multivariate orthogonal polynomials, spectral methods, partial differential equations, Eikonal equations, simplex

MSC codes. 68Q25, 68R10, 68U05

1. Global representations to local solutions of the Eikonal Equation.

Haven't really looked into the viscosity perspective, but..I think I converged on the same idea for solving the PDE. Consider the standard simplex

$$\mathcal{T}^d = \{\boldsymbol{x} = (x_1, \cdots x_d) \in \mathbb{R}^d \mid x_i \geq 0,\ 1 - |\boldsymbol{x}|_1 \geq 0\}$$

The *minimal* time $u(\boldsymbol{x})$ to reach the boundary $\partial\mathcal{T}^d$ from a point $x \in \mathcal{T}^d$ is computable through solving the Eikonal equation on $\mathcal{T}^d$:

$$\begin{cases} \sum_{i=1}^d (\partial_{x_i} u)^2 = \left(\frac{1}{f}\right)^2, & x \in \mathcal{T}^d, \\ u = q, & x \in \partial\mathcal{T}^d, \end{cases} \tag{1.1}$$

where $f : \overline{\mathcal{T}^d} \to \mathbb{R}^+_\star$ is the speed in the medium, $q : \partial\mathcal{T}^d \to \mathbb{R}^+$ is the time to leave $\mathcal{T}^d$ from a point on $\partial\mathcal{T}^d$. We seek to represent u under an optimal, finite, weighted L_2 expansion, wherein

$$u(\boldsymbol{x}) \approx \sum_{j=1}^n u_j P_j(\boldsymbol{x}) = \boldsymbol{u} \cdot \boldsymbol{P}(\boldsymbol{x}),$$

and $\boldsymbol{u}$ is the vector in $\mathbb{R}^n$ of coefficients of u under the elements of the basis $\boldsymbol{P}(\boldsymbol{x}) : \mathbb{R}^d \to \mathbb{R}^n$. Here $\approx$ must be taken in the sense of weighted L_2 convergence of the series to u as $n \to \infty$. For the typical case of $q \equiv 0$ (i.e. time to boundary is 0 if on the boundary), we employ a weighted basis which evaluates to 0 on the boundary, thereby naturally satisfying homogeneous Dirichlet conditions.

*Submitted to the editors DATE.

Funding: This work is partially supported by the National Science Foundation under Grant Number 2110886.

†Department of Applied Mathematics, University of Colorado, Boulder, CO

1.1. Jacobi bases on the Triangle. Let us specialize for $d = 2$. By P_k^n, we denote the k^{th} Jacobi polynomial on the triangle of degree n with parameters (a, b, c). When the parameters must be specified, we will refer to the same polynomial by $P_{k,n}^{(a,b,c)}$. The weight function is

$$\begin{aligned} W^{(a,b,c)}(x,y) &= x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}}, \\ w_{(a,b,c)} &= \Big(\int_{\mathcal{T}^2_{\text{ref}}} W(x,y) dx dy \Big)^{-1} = \frac{\Gamma(|\kappa| + \frac{3}{2})}{\Gamma(a+\frac{1}{2})\Gamma(b+\frac{1}{2})\Gamma(c+\frac{1}{2})} \\ w^{(a,b,c)}(\boldsymbol{x}) &= w_{(a,b,c)} W^{(a,b,c)}(\boldsymbol{x}) \end{aligned} \tag{1.2}$$

with $a, b, c > -1/2$ the Jacobi parameters, and $|\kappa| = a + b + c$. The orthonormal polynomials are given in terms of the 1D Jacobi polynomials by

$$\begin{aligned} P_k^n(x,y) &= (h_{k,n})^{-1} P_{n-k}^{2k+b+c,a-1/2}(2x-1)(1-x)^k P_k^{c-1/2,b-1/2}\Big(\frac{2y}{1-x} - 1\Big), \\ (h_{k,n})^2 &= \frac{w_{a,b,c}}{(2n + |\kappa| + \frac{1}{2})(2k+b+c)} \\ &\quad \times \frac{\Gamma(n+k+b+c+1)\Gamma(n-k+a+\frac{1}{2})\Gamma(k+b+\frac{1}{2})\Gamma(k+c+\frac{1}{2})}{(n-k)!k!\Gamma(n+k+|\kappa|+\frac{1}{2})\Gamma(k+b+c)}, \end{aligned} \tag{1.3}$$

and they satisfy a mutual orthonormality condition under the weighted inner product:

$$\langle P_{j,m}^{(a,b,c)}, P_{k,n}^{(a,b,c)} \rangle_{w^{(a,b,c)}} = \delta_{jk}\delta_{mn}. \tag{1.4}$$

A given basis polynomial is not only orthonormal to those with different total degree, but also to those others within the same-degree homogeneous subspace. In other words, P_k^n, $0 \le k \le n$ forms a basis for the space of homogeneous polynomials of degree n. By considering ordered collections of these subspaces (where ordering is in terms of the degree n of the space), instead of individual polynomials, we can more elegantly reformulate the Eikonal problem into a weak-type form. Let

$$\mathbb{P}_n(x_1, x_2) = (P_0^n(x_1,x_2), \cdots P_n^n(x_1,x_2))^T,$$

and let

$$\boldsymbol{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \cdots, \mathbb{P}_{n-1}^T)^T. \tag{1.5}$$

The above notation amounts to a vectorization of the basis for the general space of polynomials in two variables Π_{n-1}^2 defined of $\mathcal{T}^2$. Now, consider the $(n-1)$ order Jacobi polynomial expansion of the solution u under the parameter family (a, b, c). For $\boldsymbol{x} \in \mathcal{T}^d$, and $N = \dim \Pi_{n-1}^2$ coefficients $\boldsymbol{u} \in \mathbb{R}^N$, we can write it as

$$u(\boldsymbol{x}) = \boldsymbol{P}^{(a,b,c)}(\boldsymbol{x})^T \boldsymbol{u}. \tag{1.6}$$

There exists sparse discrete operators which map $\boldsymbol{u}$ to the coefficients of the derivative of u under a modified basis, as well as those which promote the basis parameters without taking derivatives:

$$\begin{aligned} \partial_{x_1} u(\boldsymbol{x}) &= \boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_1} \boldsymbol{u} \\ \partial_{x_2} u(\boldsymbol{y}) &= \boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_2} \boldsymbol{u} \end{aligned} \tag{1.7}$$

Inserting this into the fully constrained Eikonal equation gives

$$\left(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \mathcal{K}^{(a+1,b+1,c+1)}_{(a,b,c)} \boldsymbol{c}_{\frac{1}{f}}\right)^2 = \tag{1.8}$$
$$\left(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \mathcal{K}^{(a+1,b+1,c+1)}_{(a+1,b,c+1)} \mathcal{D}_{x_1}\boldsymbol{u}\right)^2 +$$
$$\left(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \mathcal{K}^{(a+1,b+1,c+1)}_{(a,b+1,c+1)} \mathcal{D}_{x_2}\boldsymbol{u}\right)^2 =$$
$$\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T \left(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}^T_{x_1} + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}^T_{x_2}\right)\boldsymbol{P}^{(a+1,b,c+1)}(\boldsymbol{x}), \tag{1.9}$$

where

$$\tilde{\mathcal{D}}_{x_1} = \mathcal{K}^{(a+1,b+1,c+1)}_{(a+1,b,c+1)}\mathcal{D}_{x_1}, \quad \tilde{\mathcal{D}}_{x_2} = \mathcal{K}^{(a+1,b+1,c+1)}_{(a,b+1,c+1)}\mathcal{D}_{x_2}, \tag{1.10}$$

and, the coefficients of $(1/f)$ are represented as $\boldsymbol{c}_{\frac{1}{f}}$ but promoted to the $(a+1, b+1, c+1)$ Jacobi basis. We let $\tilde{\cdot}$ indicate promotion *to* $(a+1, b+1, c+1)$ from any other parameter set. The above equality holds for any point $\boldsymbol{x} \in \mathcal{T}^2$. So, for a solution $\boldsymbol{u}$, the coefficient operation must be such that:

$$\left(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}^T_{x_1} + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}^T_{x_2}\right) = \tilde{\boldsymbol{c}}_{\frac{1}{f}}\tilde{\boldsymbol{c}}^T_{\frac{1}{f}}. \tag{1.11}$$

We seek a coefficient solution $\boldsymbol{u}$ which represents the solution *globally* in $\mathcal{T}^2$. However, polynomials are differentiable, and some of the problems we are interested in yield non-differentiable solutions. This is true even in the most basic case of $f \equiv 1$, in which u is the signed distance function from $\boldsymbol{x}$ to $\partial\mathcal{T}^2$, depicted below:

FIG. 1. *A simple calculation shows that the minimum distance from* $\boldsymbol{x} \in \mathcal{T}^2$ *to the boundary* $\partial\mathcal{T}^2$ *is given by the minimum of* $\left\{x_1, x_2, \left|\left|\boldsymbol{x} - \begin{bmatrix}(x_1 - x_2 + 1)/2\\(x_2 - x_1 + 1)/2\end{bmatrix}\right|\right|_2\right\}$

We cannot hope to recover a solution $\boldsymbol{u}$ which satisfies (1.11) to machine precision, as that would imply the global solution has zero residual *pointwise* everywhere. Hence,

we must return to (1.8), and consider solving it weakly, in the sense of:

(1.12)

$$\int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\boldsymbol{x})\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\tilde{\boldsymbol{c}}_{\frac{1}{f}}\tilde{\boldsymbol{c}}_{\frac{1}{f}}^T\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})d\boldsymbol{x} \approx ||1/f||^2_{w_{(a+1,b+1,c+1)}}$$
$$\approx \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\boldsymbol{x})\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\big(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_2}^T\big)\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})d\boldsymbol{x}$$
$$\approx ||\partial_x u||^2_{w^{(a+1,b+1,c+1)}} + ||\partial_y u||^2_{w^{(a+1,b+1,c+1)}}.$$ ∎

Here, the norm is viewed as that of a function in $L^2(\mathcal{T}^2, w^{(a+1,b+1,c+1)})$. Parsing this equation out a bit further (condensing sub/super scripts), we have

$$\int_{\mathcal{T}^2}\boldsymbol{P}(\boldsymbol{x})^T(\boldsymbol{u_x}\boldsymbol{u_x}^T + \boldsymbol{u_y}\boldsymbol{u_y}^T)w(\boldsymbol{x})\boldsymbol{P}(\boldsymbol{x})d\boldsymbol{x} \quad (\textbf{Only } x \textbf{ part handled below})$$
$$= \int_{\mathcal{T}^2}\boldsymbol{P}(\boldsymbol{x})^T\begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix}\boldsymbol{P}(\boldsymbol{x})w(\boldsymbol{x})d\boldsymbol{x}$$
$$= \int_{\mathcal{T}^2}\begin{bmatrix}P_1(\boldsymbol{x}) & P_2(\boldsymbol{x}) & \cdots & P_N(\boldsymbol{x})\end{bmatrix}\begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix}\begin{bmatrix}P_1(\boldsymbol{x}) \\ P_2(\boldsymbol{x}) \\ \vdots \\ P_N(\boldsymbol{x})\end{bmatrix}w(\boldsymbol{x})d\boldsymbol{x}$$
$$= \int_{\mathcal{T}^2}\begin{bmatrix}P_1(\boldsymbol{x}) & P_2(\boldsymbol{x}) & \cdots & P_N(\boldsymbol{x})\end{bmatrix}\begin{bmatrix} u_{x,1}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \\ u_{x,2}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \\ \vdots \\ u_{x,N}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \end{bmatrix}w(\boldsymbol{x})d\boldsymbol{x}$$
$$= \int_{\mathcal{T}^2} w(\boldsymbol{x})\sum_{k=1}^N P_k(\boldsymbol{x})u_{x,k}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x})d\boldsymbol{x} = \int_{\mathcal{T}^2} w(\boldsymbol{x})\sum_{k=1}^N\sum_{j=1}^N u_{x,k}u_{x,j}P_k(\boldsymbol{x})P_j(\boldsymbol{x})d\boldsymbol{x}$$
$$= \int_{\mathcal{T}^2}\sum_{k=1}^N u_{x,k}^2P_k^2(\boldsymbol{x})w(\boldsymbol{x})d\boldsymbol{x} = \sum_{k=1}^N u_{x,k}^2\int_{\mathcal{T}^2}P_k^2(\boldsymbol{x})dw(\boldsymbol{x}) \quad (\textbf{ now we add back in } y \textbf{ part })$$
$$= \sum_{k=1}^N u_{x,k}^2 + \sum_{k=1}^N u_{y,k}^2$$
$$= \boldsymbol{u}^T(\tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y)\boldsymbol{u} = \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T\tilde{\boldsymbol{c}}_{\frac{1}{f}},$$ ∎

where we've used the mutual orthonormality relations, which persist even within a subspace of homogeneous polynomials of degree m, to eliminate all off-diagonal terms in the sum. Note, not all orthonormal bases are created equal, and we normally would have to retain many outer product terms in the sum because orthonormality between vectors of the same subspace is not a given.

Thus, we want to solve the following quadratic program with equality constraints:

$$\min_{\boldsymbol{u}\in\mathbb{R}^N} F(\boldsymbol{u}) = \boldsymbol{u}^T G\boldsymbol{u} - \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T \tilde{\boldsymbol{c}}_{\frac{1}{f}} \tag{1.13}$$
$$\text{subject to} \quad \boldsymbol{u}\cdot\boldsymbol{P}(\boldsymbol{x}) = 0, \quad \boldsymbol{x}\in\partial\Omega,$$

where $G = \tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite [1]. The gradient and Hessian of F are given by

$$\nabla_{\boldsymbol{u}} F(\boldsymbol{u}) = 2G\boldsymbol{u}, \tag{1.14}$$
$$\nabla^2_{\boldsymbol{u}} F(\boldsymbol{u}) = 2G, \tag{1.15}$$

and we still need to consider the constraint equation. We take the approach of eliminating the constraints by solving a the minimization problem on auxiliary variables. That is, suppose $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^{M\times N}$, $M > N$, has rank p. We can compute the *full SVD* of $\boldsymbol{P}(\boldsymbol{x}) = U\Sigma V^T$. Then, the last $n-p$ rows of V^T form a basis for the $\mathcal{N}(\boldsymbol{P}(\boldsymbol{x}))$. That is, $V^T = \begin{bmatrix} V_p^T \\ V_{n-p}^T \end{bmatrix}$, and any solution $\boldsymbol{u}$ satisfying the equality constraints can be expressed as

$$\boldsymbol{u} = V_{n-p}\boldsymbol{u}_{\text{eq}}, \tag{1.16}$$

Hence, we can solve

$$\min_{\boldsymbol{u}_{\text{eq}}\in\mathbb{R}^N} F(\boldsymbol{u}_{\text{eq}}) = \boldsymbol{u}_{\text{eq}}^T V_{n-p}^T G V_{n-p}\boldsymbol{u}_{\text{eq}} - \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T \tilde{\boldsymbol{c}}_{\frac{1}{f}} \tag{1.17}$$

and finally find $\boldsymbol{u}$ by computing as in (1.16). Note, the gradient and Hessian of the auxiliary problem are given by

$$\nabla_{\boldsymbol{u}_{\text{eq}}} F(\boldsymbol{u}_{\text{eq}}) = 2V_{n-p}^T G V_{n-p}\boldsymbol{u}_{\text{eq}}, \tag{1.18}$$
$$\nabla^2_{\boldsymbol{u}} F(\boldsymbol{u}) = 2V_{n-p}^T G V_{n-p}. \tag{1.19}$$

Note, we can find p by the number of singular values above some small threshold that we choose (near $\varepsilon_{\text{mach}}$) so that it represents numerical rank.

Remark 1.1. There is a ton of material in chapter 12 and 16 of the Nocedal/Wright optimization book. I believe I can prove properties on minimizers to (1.13) (though unnecessary) and maybe use a direct method therein. There still remains the problem of initialization..perhaps we just initialize from the null space, or solve a linearized problem?

Remark 1.2. By making use of a weighted basis $\tilde{\boldsymbol{P}}(\boldsymbol{x})$, and expressing our solution coefficients and differential operators in terms of this weighted basis, we can alternatively eliminate the constraint equation and solve a completely unconstrained quadratic optimization problem I think I need to do some more symbolic computation to figure out the form of the RHS and/or additional conditions u must first satisfy so that we solve the same problem. I implemented that weighted basis, but I don't think I will get things to work properly, or rather, I will not continue trying.We provide more details in the next section.

[1] Note, $\forall \boldsymbol{x}\in\mathbb{R}_\star^{n^+}, \boldsymbol{x}^T G\boldsymbol{x} = \boldsymbol{x}^T\tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x\boldsymbol{x} + \boldsymbol{x}^T\tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y\boldsymbol{x} = ||\tilde{\mathcal{D}}_x\boldsymbol{x}||^2 + ||\tilde{\mathcal{D}}_y\boldsymbol{x}||^2 \geq 0$. We have equality only for any $\boldsymbol{x} = \alpha\boldsymbol{e}_1$, as derivatives of the constant polynomial are 0.

1.2. A variational approach. Let's try framing the problem from the perspective of infinite dimensional optimization. Thus far, my efforts have fallen under the discretize-then-optimize approach for solving non-linear PDEs. So, let's flip that and consider the optimize-then-discretize approach. The first order conditions for optimality of the functional may serve as a way to initialize the problem for the discretized non-linear optimization call. We want to solve

$$\begin{aligned} &\min_{u\in L_2(\mathcal{T}^2,w^{(a,b,c)})} \mathcal{L}(u), \\ &\text{s.t. } u = q \text{ on } \partial\mathcal{T}^2 \end{aligned}, \tag{1.20}$$

where $\mathcal{L} : L_2(\mathcal{T}^2, w^{(a,bc)}) \to \mathbb{R}^+$ is defined by

$$\mathcal{L}(u) := \int_{\mathcal{T}^2} \Big[|\nabla_{\boldsymbol{x}} u|^2 - \big(\frac{1}{f}\big)^2\Big]^{\frac{1}{2}} d\boldsymbol{x}, \tag{1.21}$$

and u, f are as in (1.1), restricted to $L_2(\mathcal{T}^2, \mathcal{W}^{(a,bc)})$. The steps here are to first take the Gateaux derivative of $\mathcal{L}$ as

$$\delta\mathcal{L}(u,\hat{u}) = \partial\varepsilon\mathcal{L}(u+\varepsilon\hat{u})|_{\varepsilon=0} = 0,$$

where $\hat{u} \in \mathcal{H}^1_0(\mathcal{T}^2)$. This gives the necessary condition a minimizer of $\mathcal{L}$ must satisfy. Before proceeding, recall Green's identity for $u, v \in H^1(\mathcal{T}^2)$, i.e. a multidimensional integration by parts formula:

$$\int_{\mathcal{T}^2} \nabla_{\boldsymbol{x}} u \cdot v d(\boldsymbol{x}) = \int_{\partial\mathcal{T}^2} (uv)\cdot \boldsymbol{n} dS(\boldsymbol{x}) - \int_{\mathcal{T}^2} u\nabla_{\boldsymbol{x}} \cdot v d(\boldsymbol{x}). \tag{1.22}$$

Accordingly, we have

$$\begin{aligned} 0 &= \partial_\varepsilon \mathcal{L}(u+\varepsilon\hat{u})|_{\varepsilon=0} \\ &= \partial_\epsilon \Big[\int_{\mathcal{T}^2} \Big((u_x^2+u_y^2) + 2\varepsilon(\hat{u}_x u_x + \hat{u}_y u_y) + \varepsilon^2(\hat{u}_x^2+\hat{u}_y^2) - \big(\frac{1}{f}\big)^2\Big)^{\frac{1}{2}} d(\boldsymbol{x})\Big]\Big|_{\varepsilon=0} \\ &= \Big[\frac{1}{2}\int_{\mathcal{T}^2} \frac{\Big(2(\hat{u}_x u_x + \hat{u}_y u_y) + 2\varepsilon(\hat{u}_x^2+\hat{u}_y^2)\Big)}{\Big((u_x^2+u_y^2) + 2\varepsilon(\hat{u}_x u_x + \hat{u}_y u_y) + \varepsilon^2(\hat{u}_x^2+\hat{u}_y^2) - \big(\frac{1}{f}\big)^2\Big)^{\frac{1}{2}}} d(\boldsymbol{x})\Big]\Big|_{\varepsilon=0}. \end{aligned}$$

The first variation is then given by

$$\delta\mathcal{L}(u,\hat{u}) = \int_{\mathcal{T}^2} \frac{\nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}}\hat{u}}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - (\frac{1}{f})^2}} d(\boldsymbol{x}). \tag{1.23}$$

We can apply (1.22) with $u \leftarrow \hat{u}$ and $v \leftarrow \frac{\nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}}$ to derive the strong (PDE) form of the first order necessary condition a minimizer of $\mathcal{L}$ must satisfy. Using the fact that $\hat{u}$ is 0 on the boundary, we have

$$\begin{aligned} \delta\mathcal{L}(u,\hat{u}) &= \int_{\partial\mathcal{T}^2} \hat{u}\frac{\nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}} \cdot \boldsymbol{n} dS(\boldsymbol{x}) - \int_{\mathcal{T}^2} \hat{u}\nabla_{\boldsymbol{x}} \cdot \frac{\nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}} d(\boldsymbol{x}) \\ &= -\int_{\mathcal{T}^2} \hat{u}\nabla_{\boldsymbol{x}} \cdot \frac{\nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}} d(\boldsymbol{x}) = 0 \end{aligned} \tag{1.24}$$

Furthermore, the choice of $\hat{u}$ is arbitrary. Thus, we find that a minimizer u satisfies the following PDE in $\mathcal{T}^2$;

$$
\begin{aligned}
-\nabla_{\boldsymbol{x}} \cdot \left(\frac{\nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}} \right) = 0, \quad & \boldsymbol{x} \in \mathcal{T}^2, \\
u = q, \quad & \boldsymbol{x} \in \partial \mathcal{T}^2.
\end{aligned} \tag{1.25}
$$

Next, we compute the Frechet derivative $\delta^2 \mathcal{L}(u, \hat{u}, \tilde{u})$, where $\tilde{u} \in H_0^1(\mathcal{T}^2)$. Letting $u \leftarrow u + \varepsilon \tilde{u}$ in (1.23), we have

$$
\begin{aligned}
\delta^2 \mathcal{L}(u, \hat{u}, \tilde{u}) &= \partial_\varepsilon \delta \mathcal{L}(u + \varepsilon \tilde{u}, \hat{u}) \Big|_{\varepsilon = 0} \\
&= \left(\partial_\varepsilon \int_{\mathcal{T}^2} \frac{\nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}} \hat{u} + \varepsilon \nabla_{\boldsymbol{x}} \tilde{u} \cdot \nabla_{\boldsymbol{x}} \hat{u}}{\sqrt{-\frac{1}{f^2} + \nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}} u + 2\varepsilon \nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}} \tilde{u} + \varepsilon^2 \nabla_{\boldsymbol{x}} \tilde{u} \cdot \nabla_{\boldsymbol{x}} \tilde{u}}} d(\boldsymbol{x}) \right) \Big|_{\varepsilon = 0} \\
&= \int_{\mathcal{T}^2} \left(\frac{\nabla_{\boldsymbol{x}} \tilde{u} \cdot \nabla_{\boldsymbol{x}} \hat{u}}{\sqrt{-\frac{1}{f^2} + |\nabla_{\boldsymbol{x}} u|^2}} - \frac{(\nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}} \hat{u})(\nabla_{\boldsymbol{x}} u \cdot \nabla_{\boldsymbol{x}} \tilde{u})}{(-\frac{1}{f^2} + |\nabla_{\boldsymbol{x}} u|^2)^{\frac{3}{2}}} \right) d(\boldsymbol{x})
\end{aligned} \tag{1.26}
$$

The weak form of the Newton system at a point $u^k \in H_0^1(\mathcal{T}^2)$ is then independent of $\hat{u}$, and given by

$$
\begin{aligned}
&\text{Find } \tilde{u} \in H_0^1(\mathcal{T}^2) \text{ s.t.} \\
&\delta^2 \mathcal{L}(u^k)(\hat{u}, \tilde{u}) = -\delta \mathcal{L}(u^k)(\hat{u}), \\
&\text{and set } u^{k+1} = u^k + \tilde{u}, \\
&\text{with } u^0 = q \text{ on } \partial \mathcal{T}^2.
\end{aligned} \tag{1.27}
$$

To derive the strong form of the Newton system, we proceed as for the first variation using integration by parts formula. Let $u \leftarrow \hat{u}$ and $v \leftarrow \frac{\nabla_{\boldsymbol{x}} \tilde{u}}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}}$ for the first term in (1.26). For the second term therein, let $u \leftarrow \hat{u}$ and $v \leftarrow \frac{\nabla_{\boldsymbol{x}} \tilde{u} |\nabla_{\boldsymbol{x}} u|^2}{(|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2})^{\frac{3}{2}}}$. Using the fact that $\hat{u} \in H_0^1(\mathcal{T}^2)$, and incorporating the first variation from (1.23) as needed in (1.27), we find the Newton system is equivalently expressed as

$$
\int_{\mathcal{T}^2} \hat{u} \Big[\nabla_{\boldsymbol{x}} \cdot \Big(\frac{\nabla_{\boldsymbol{x}} \tilde{u} |\nabla_{\boldsymbol{x}} u|^2}{(|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2})^{\frac{3}{2}}} - \frac{\nabla_{\boldsymbol{x}} \tilde{u} + \nabla_{\boldsymbol{x}} u}{\sqrt{|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2}}} \Big) \Big] d(\boldsymbol{x}) = 0
$$

Then, by the arbitrariness of $\hat{u}$, we find the strong form of the Newton system is

$$
\begin{aligned}
&\text{Find } \tilde{u} \in H_0^1(\mathcal{T}^2) \text{ s.t.} \\
\nabla_{\boldsymbol{x}} \cdot \Big[\nabla_{\boldsymbol{x}} \tilde{u} \Big(\frac{|\nabla_{\boldsymbol{x}} u^k|^2}{(|\nabla_{\boldsymbol{x}} u^k|^2 - \frac{1}{f^2})^{\frac{3}{2}}} - \frac{1}{\sqrt{|\nabla_{\boldsymbol{x}} u^k|^2 - \frac{1}{f^2}}} \Big) \Big] &= \nabla_{\boldsymbol{x}} \cdot \Big(\frac{\nabla_{\boldsymbol{x}} u^k}{\sqrt{|\nabla_{\boldsymbol{x}} u^k|^2 - \frac{1}{f^2}}} \Big) \\
&\text{and set } u^{k+1} = u^k + \tilde{u}, \\
&\text{with } u^0 = q \text{ on } \partial \mathcal{T}^2.
\end{aligned} \tag{1.28}
$$

By choosing $u^0 = q$ on $\partial\mathcal{T}^2$, and setting $u^{k+1} = u^k + \tilde{u}$, we have an infinite-dimensional newton stepper for solving the Eikonal problem. To avoid the trivial solution $u \equiv 0$ when $q \equiv 0$, we can try this with

$$u^0 = w^{(3/2,3/2,3/2)}(x, y) = xy(1 - x - y),$$

and use the weighted basis for differentiation in coefficient space. We iterate until the relative step difference is within some tolerance, in the $L_2(\mathcal{T}^2, w)$ norm. I will give details on the weighted Laplace operator momentarily.

Algorithm 1.1 Netwon updates for solving (1.1)

Require: `tol` < 1
  $u^k \leftarrow xy(1 - x - y)$
  $\tilde{u} \leftarrow 1$
  while $\big(|\nabla_{\boldsymbol{x}} u^k|^2 - (\frac{1}{f})^2 > 0\big)$ and $||\tilde{u}||_{L^2(\mathcal{T}^2,\tilde{w})} >$ `tol` **do**
    $\tilde{u} \leftarrow$ solution to (1.28)
    $u^{k+1} \leftarrow u^k + \tilde{u}$
    $u^k \leftarrow u^{k+1}$
  end while

Note, Algorithm 1.1 as well as the preceding derivations of weak and strong forms all assume that $|\nabla_{\boldsymbol{x}} u^k|^2 - (\frac{1}{f})^2 > 0$ for any k. To prevent the need for this check, we can consider the modified (though equivalent) functional

$$\mathcal{L}(u) = \int_{\mathcal{T}^2} [(|\nabla_{\boldsymbol{x}} u|^2 - \frac{1}{f^2})^2]^{\frac{1}{4}} \tag{1.29}$$

Even though (1.29) is equivalent to (1.21), the latter does not necessarily ensure that the range of $\mathcal{L}$ is $\mathbb{R}^+$ without specifying the appropriate branch of $\sqrt{}$ to consider. The modified functional does this without us having to deal with absolute values or the sgn function. Any arguments raised to the $\frac{1}{2}$ or $\frac{3}{2}$ powers, as they are in (1.28), can simply be squared and square root-ed before exponentiation. That is, the numerical implementation will include such a transformation to those arguments (this might work? we'll see). Expressing (1.28) in modal form is our next challenge.

2. The weighted Jacobi basis. The weighted basis

$$\tilde{\boldsymbol{P}}^{(a,b,c)}(x,y) = x^{a-\frac{1}{2}}y^{b-\frac{1}{2}}(1-x-y)^{c-\frac{1}{2}}\boldsymbol{P}^{(a,b,c)} \tag{2.1}$$

will prove useful for incorporating boundary conditions to the discrete operator, while preserving its sparsity. Note, we are simply multiplying the original basis by the weight function $W^{(a,b,c)}(\boldsymbol{x})$ which forms the measure under which they are orthogonal. We need the following weighted ladder operators for differentiation in the weighted basis:

$$\begin{aligned} -(2k+b+c+1)h_{k,n}^{(a,b,c)}\partial_x\tilde{P}_{k,n}^{(a,b,c)} &= (k+c)(n-k+1)h_{k,n+1}^{(a-1,b,c-1)}\tilde{P}_{k,n+1}^{(a-1,b,c-1)} \\ &\quad + (k+1)(n-k+a)h_{k+1,n+1}^{(a-1,b,c-1)}\tilde{P}_{k+1,n+1}^{(a-1,b,c-1)} \end{aligned} \tag{2.2}$$

$$h_{k,n}^{(a,b,c)}\partial_y\tilde{P}_{k,n}^{(a,b,c)} = -(k+1)h_{k+1,n+1}^{(a,b-1,c-1)}\tilde{P}_{k+1,n+1}^{(a,b-1,c-1)}. \tag{2.3}$$

If $\boldsymbol{f}$ are the coefficients of f under the (a,b,c) basis, then we have sparse banded matrices $\mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)}, \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)}$ such that

$$\begin{aligned} \partial_x[x^{a-\frac{1}{2}}y^{b-\frac{1}{2}}(1-x-y)^{c-\frac{1}{2}}f(x,y)] &= \partial_x(\tilde{\boldsymbol{P}}^{(a,b,c)}(x,y)^T\boldsymbol{f}) \\ &= \tilde{\boldsymbol{P}}^{(a-1,b,c-1)}(x,y)^T\mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)}\boldsymbol{f}, \end{aligned} \tag{2.4}$$

$$\begin{aligned} \partial_y[x^{a-\frac{1}{2}}y^{b-\frac{1}{2}}(1-x-y)^{c-\frac{1}{2}}f(x,y)] &= \partial_y(\tilde{\boldsymbol{P}}^{(a,b,c)}(x,y)^T\boldsymbol{f}) \\ &= \tilde{\boldsymbol{P}}^{(a,b-1,c-1)}(x,y)^T\mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)}\boldsymbol{f} \end{aligned} \tag{2.5}$$

2.0.1. Lowering operators. Additionally, we have lowering operators which relate $xP_{k,n}^{(a,b,c)}$ to $P_{k,n}^{(a-1,b,c)}$ and $yP_{k,n}^{(a,b,c)}$ to $P_{k,n}^{(a,b-1,c)}$. These are given by the following recurrences:

$$\begin{aligned} (2n+a+b+c+2)h_{k,n}^{(a,b,c)}xP_{k,n}^{(a,b,c)} &= (n-k+a)h_{k,n}^{(a-1,b,c)}P_{k,n}^{(a-1,b,c)} \\ &\quad + (n-k+1)h_{k,n+1}^{(a-1,b,c)}P_{k,n+1}^{(a-1,b,c)}, \end{aligned} \tag{2.6}$$

$$\begin{aligned} (2k+b+c+1)(2n+a+b+c+2)h_{k,n}^{(a,b,c)}yP_{k,n}^{(a,b,c)} &= (k+b)(n+k+b+c+1)h_{k,n}^{(a,b-1,c)}P_{k,n}^{(a,b-1,c)} \\ &\quad - (k+1)(n-k+a)h_{k+1,n}^{(a,b-1,c)}P_{k+1,n}^{(a,b-1,c)} \\ &\quad - (k+b)(n-k+1)h_{k,n+1}^{(a,b-1,c)}P_{k,n+1}^{(a,b-1,c)} \\ &\quad + (k+1)(n+k+a+b+c+2)h_{k+1,n+1}^{(a,b-1,c)}P_{k+1,n+1}^{(a,b-1,c)}. \end{aligned} \tag{2.7}$$

So, there are sparse banded matrices $\mathcal{L}_{(a,b,c)}^{(a-1,b,c)}$ and $\mathcal{L}_{(a,b,c)}^{(a,b-1,c)}$ corresponding to multiplication by x and y, respectively, such that

$$xf(x,y) = x\boldsymbol{P}^{(a,b,c)}(x,y)^T\boldsymbol{f} = \boldsymbol{P}^{(a-1,b,c)}(x,y)^T\mathcal{L}_{(a,b,c)}^{(a-1,b,c)}\boldsymbol{f}, \tag{2.8}$$

$$yf(x,y) = y\boldsymbol{P}^{(a,b,c)}(x,y)^T\boldsymbol{f} = \boldsymbol{P}^{(a,b-1,c)}(x,y)^T\mathcal{L}_{(a,b,c)}^{(a,b-1,c)}\boldsymbol{f}. \tag{2.9}$$

2.0.2. Discrete differential operators. We will construct the modal Eikonal operator which operates on coefficients under the basis $\tilde{\boldsymbol{P}}^{(3/2,3/2,3/2)}$ and produces coefficients under $\boldsymbol{P}^{(3/2,3/2,3/2)}$. Note, *the conversion from the weighted to regular*

Jacobi basis depends on our choice of parameters $a = b = c = 3/2$. With $\boldsymbol{f}$ the coefficients of $f(x,y)$ in the (a,b,c) basis,

$$\begin{aligned}
\partial_x[W^{(a,b,c)}(x,y)f(x,y)] &= \tilde{\boldsymbol{P}}^{(a-1,b,c-1)}(x,y)^T \mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \boldsymbol{f} \\
&= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \left(y\boldsymbol{P}^{(a-1,b,c-1)}(x,y)\right)^T (\mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \boldsymbol{f}) \\
&= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \boldsymbol{P}^{(a-1,b-1,c-1)}(x,y)^T (\mathcal{L}^{(a-1,b-1,c-1)}_{(a-1,b,c-1)} \mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \boldsymbol{f}), \\
\partial_y[W^{(a,b,c)}(x,y)f(x,y)] &= \tilde{\boldsymbol{P}}^{(a,b-1,c-1)}(x,y)^T \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f} \\
&= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \left(x\boldsymbol{P}^{(a,b-1,c-1)}(x,y)\right)^T (\mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f}) \\
&= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \boldsymbol{P}^{(a-1,b-1,c-1)}(x,y)^T (\mathcal{L}^{(a-1,b-1,c-1)}_{(a,b-1,c-1)} \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f})
\end{aligned}$$

When $a = b = c = 3/2$, we have

$$\begin{aligned}
\partial_x[W^{(a,b,c)}(x,y)f(x,y)] &= \boldsymbol{P}^{(a-1,b-1,c-1)}(x,y)^T (\mathcal{L}^{(a-1,b-1,c-1)}_{(a-1,b,c-1)} \mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \boldsymbol{f}), \\
\partial_y[W^{(a,b,c)}(x,y)f(x,y)] &= \boldsymbol{P}^{(a-1,b-1,c-1)}(x,y)^T (\mathcal{L}^{(a-1,b-1,c-1)}_{(a,b-1,c-1)} \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f}),
\end{aligned}$$

We need to see what auxiliary problems we must solve, as the nonlinearity of (1.1) leads to cross-terms that must cancel if we hope that a solution to the weighted problem also solves the unweighted problem. In the bulk, we have

$$[\partial_x(Wu)]^2 + [\partial_y(Wu)]^2 = W^2[u_x^2 + u_y^2] + u^2(W_x + W_y) + 2uW(W_x u_x + W_y u_y),$$

and we need the second two terms in the equation to cancel. There's something here we can do by integrating, and finding the additional conditions u must satisfy. I think it would become a solution for the initial guess of u. Essentially, we know $W^2[u_x^2 + u_y^2] = W^2/f^2$, since we assume that u solve (1.1). Then, we can enforce that

$$\int_{\mathcal{T}^2} u^2(W_x + W_y) + 2uW(W_x u_x + W_y u_y) = 0.$$

There may be some simplification that occurs if we plug in the Jacobi expansion of u .. idk

In the weighted basis, second partials acting on function modes can be constructed with the promotion and unweighted ladder operator as done in Section 2.0.2.

(2.11)
$$\partial_{xx} f(x,y) = \boldsymbol{P}^{(a,b,c)}(x,y)^T \left(\mathcal{K}^{(a,b,c)}_{(a,b-1,c)} \mathcal{D}^{(a,b-1,c)}_{x,(a-1,b-1,c-1)} \mathcal{L}^{(a-1,b-1,c-1)}_{(a-1,b,c-1)} \mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \boldsymbol{f}\right),$$

(2.12)
$$\partial_{yy} f(x,y) = \boldsymbol{P}^{(a,b,c)}(x,y)^T \left(\mathcal{K}^{(a,b,c)}_{(a-1,b,c)} \mathcal{D}^{(a-1,b,c)}_{y,(a-1,b-1,c-1)} \mathcal{L}^{(a-1,b-1,c-1)}_{(a,b-1,c-1)} \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f}\right),$$

(2.13)
$$\partial_{xy} f(x,y) = \boldsymbol{P}^{(a,b,c)}(x,y)^T \left(\mathcal{K}^{(a,b,c)}_{(a,b-1,c)} \mathcal{D}^{(a,b-1,c)}_{x,(a-1,b-1,c-1)} \mathcal{L}^{(a-1,b-1,c-1)}_{(a,b-1,c-1)} \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)} \boldsymbol{f}\right)$$

Δ_W, the weighted Laplace operator, is then given by

(2.14)
$$\begin{aligned}
\Delta_W = &\mathcal{K}^{(a,b,c)}_{(a,b-1,c)} \mathcal{D}^{(a,b-1,c)}_{x,(a-1,b-1,c-1)} \mathcal{L}^{(a-1,b-1,c-1)}_{(a-1,b,c-1)} \mathcal{W}^{(a-1,b,c-1)}_{x,(a,b,c)} \\
&+ \mathcal{K}^{(a,b,c)}_{(a-1,b,c)} \mathcal{D}^{(a-1,b,c)}_{y,(a-1,b-1,c-1)} \mathcal{L}^{(a-1,b-1,c-1)}_{(a,b-1,c-1)} \mathcal{W}^{(a,b-1,c-1)}_{y,(a,b,c)}.
\end{aligned}$$

The cross partial terms are needed only when we include affine transformations from general triangles to the standard reference triangle, as they appear when chaining out derivatives of the parametrization.

3. Quadrature generation for Jacobi polynomials on the Simplex. Here, we detail the procedure for generating quadrature rules on the simplex. Suppose that $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^N$, with

$$N = \dim(\Pi_{n-1}^d) = \binom{n-1+d}{n-1},$$

as in (1.5) (in which we had $d = 2$), and $\boldsymbol{x} \in \mathbb{R}^d$. The orthonormal 3-term relation for Jacobi polynomials on the simplex is given by

$$x_i \mathbb{P}_n = A_{n,i}\mathbb{P}_{n+1} + B_{n,i}\mathbb{P}_n + A_{n-1,i}^T \mathbb{P}_{n-1},\ i = 1, \cdots d \tag{3.1}$$

$$A_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_{n+1}^T \rangle \in \mathbb{R}^{r_n^d \times r_{n+1}^d}, \tag{3.2}$$

$$B_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_n^T \rangle \in \mathbb{R}^{r_n^d \times r_n^d}, \tag{3.3}$$

where $\langle \cdot \rangle$ is the weighted L^2 inner product. The matrix recurrence generates an analog of the Jacobi matrix from 1D, except now there are d of them - one for each dimension:

$$\mathcal{J}_{n,i} = \begin{bmatrix} B_{0,i} & A_{0,i} & & & \\ A_{0,i}^T & B_{1,i} & A_{1,i} & & \\ & A_{1,i}^T & \ddots & \ddots & \\ & & \ddots & B_{n-2,i} & A_{n-2,i} \\ & & & A_{n-2,i}^T & B_{n-1,i} \end{bmatrix},\ i = 1, \cdots d \tag{3.4}$$

with $\mathcal{J}_{n,i} \in \mathbb{R}^{N \times N}$ symmetric. By definition, we see that the Jacobi matrices satisfy

$$x_i \boldsymbol{P} = \mathcal{J}_{n,i}\boldsymbol{P} + \begin{bmatrix} \mathbf{0}_{(\mathbf{N}-\mathbf{n})\times \mathbf{1}} \\ A_{n-1,i}\mathbb{P}_n \end{bmatrix}, \quad i = 1, 2. \tag{3.5}$$

We hope for a way to generalize the univariate Golub-Welsch algorithm for determining Gaussian quadrature from orthogonal polynomials on the line to the multivariate case on the simplex. We say that $\Lambda = (\lambda_1, \cdots, \lambda_d)$ is a *joint eigenvalue* of $J_{n,1}, \cdots, J_{n,d}$ if there is a vector $\xi \neq 0$ such that $J_{n,i}\xi = \lambda_i \xi$ for $i = 1, \cdots, d$. The vector ξ is the *joint eigenvector*. If there are N distinct joint eigenvalues, then the weights and abscissa for the Gaussian quadrature are as in 1D - the nodes are the joint eigenvalues, and the weights are the square of the first component of the corresponding joint eigenvector. However, Theorem 3.7.5 of Dunkl-Xu states that a necessary and sufficient condition for this to occur is the following commutativity relation

$$A_{n-1,i}A_{n-1,j}^T = A_{n-1,j}A_{n-1,i}^T,\ 1 \leq i, j \leq d \tag{3.6}$$

This is equivalent to stating that $J_{n,i}$ are a commuting family, and hence, simultaneously diagonalizable. By inspection, we see that the coefficient matrices in the recurrence for Jacobi polynomials on the triangle do not, in general, satisfy the commutativity relation. Thus, *there do not exist minimal Gaussian quadrature rules on the Triangle using the classic orthogonal polynomials.*

We need an accurate quadrature on the triangle, as the convergence of any modal solver depends highly on accurate representations of the right-hand-side in the modal basis. While $J_{n,i}$ may not commute, there is a connection between their spectrums and non-minimal Gaussian-like quadrature rules on $\mathcal{T}^d$. Vioreanu and Rokhlin show that the joint eigenvalues of the combined complex operator

$$\mathcal{J}_n = \mathcal{J}_{n,1} + \iota \mathcal{J}_{n,2} \tag{3.7}$$

lie within $\mathcal{T}^2$ (Theorem 3.4 in VRQUAD). The algorithm described in their work uses the constant unit weight function and gives an alternate way to evaluate $\mathcal{J}_n$. For the case of the weighted space $L^2(\mathcal{T}, w_{a,b,c}W(x,y))$ with normalized weight (see (1.2)), the entries are

$$(\mathcal{J}_n)_{i,j} = \int_{\mathcal{T}} (x_1 + \iota x_2) P_j(x,y) P_i(x,y) w_{a,b,c} W(x,y) dxdy, \quad 0 \le i,j \le N-1, \tag{3.8}$$

where P_j is the j^{th} component of $\boldsymbol{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \cdots, \mathbb{P}_{n-1}^T)^T = (P_0^0, P_0^1, P_1^1, \cdots P_n^n)^T \in \mathbb{R}^N$. These are precisely the same operators defined in (3.7). However, this "complexification" trick is not available to us in dimensions $d > 2$, but there do exist algorithms for finding *approximate* joint diagonalizations of non-commuting matrix families.

3.1. Approximate joint diagonalization of non-commuting symmetric matrices. The approximate joint diagonalization problem for a set of real-valued symmetric matrices $\{C^1, \cdots, C^d\}$ is to find the orthogonal matrix U such that the matrices

$$F^k = UC^kU^T, \quad 1 \le k \le d \tag{3.9}$$

are approximately diagonal. Alternatively, we can find the diagonal matrices F^k such that C^k are approximately equal to $U^T F^k U$. The definition of "as diagonal as possible" leads to various formulations of the diagonalization problem in terms of minimizing an objective function. For example, solving

$$\min_{\substack{U \in \mathbb{R}^N, \\ U^TU = UU^T = I}} \sum_{i \ne j} (F_{ij}^k)^2 \tag{3.10}$$

is equivalent to minimizing the off-diagonal entries of F^k by orthogonal similarity transformations of C^k. On the other hand, solving

$$\min_{\substack{U, F^1, \cdots F^d, \\ U^TU = UU^T = I}} \sum_{k=1}^{d} \left|\left|C^k - U^T F^k U\right|\right|_F^2 \tag{3.11}$$

is finding the best representation of C^k in the basis of U - a subspace fitting approach. For both problems, we require orthogonality of U, as we want it to represent the "average eigenstructure" of C^k. For the Jacobi matrices $J_{n,i}$, this requirement is natural. We want a decomposition close to the optimal quadrature case of exact joint diagonlization. For this purpose, we use the algorithm specified in "Jacobi Angles for Simultaneous Diagonalization". The joint-eigenvalues provide us with an initial guess to a subsequent optimization problem.

3.2. Strategy for finding optimal quadrature on the Simplex. We want to integrate the M polynomials up to total degree $m-1$ exactly with $m \ge n$ using only N nodes and weights. That is, with $M = \dim(\Pi_{m-1}^d) = \binom{m-1+d}{m-1}$, the integrals for which we want an exact quadrature are

$$\begin{aligned} \mathcal{I}_j &= \int_{\mathcal{T}^d} P_j(\boldsymbol{x}) w_{\boldsymbol{a}} W(\boldsymbol{x}) d(\boldsymbol{x}), \quad 0 \le j \le M-1 \\ &= \langle P_j, P_0 \rangle = \delta_{j0}, \end{aligned} \tag{3.12}$$

where we use mutual orthonormality under $w_{a,b,c}W(x,y)$ (1.4) to evaluate them and arrive at the δ_{j0} term. If $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_d, \boldsymbol{W} \in \mathbb{R}^N$ are the nodes and weights, $\boldsymbol{\mathcal{I}} \in \mathbb{R}^M$

the above integrals, and $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^M$ (with $\boldsymbol{x} \in \mathbb{R}^d$) our M polynomials, the objective function we aim to find a zero of is $\boldsymbol{F} : \mathbb{R}^{(d+1)N} \to \mathbb{R}^M$ given by

$$\boldsymbol{F}(\boldsymbol{X}_1, \cdots, \boldsymbol{X}_d, \boldsymbol{W}) = \boldsymbol{P}(\boldsymbol{X})\boldsymbol{W} - \boldsymbol{\mathcal{I}}, \quad \boldsymbol{\mathcal{I}} \in \mathbb{R}^M, \ \boldsymbol{P}(\boldsymbol{X}) \in \mathbb{R}^{M \times N} \tag{3.13}$$

Letting

$$\boldsymbol{z} = (\boldsymbol{X}_1, \cdots, \boldsymbol{X}_d, \boldsymbol{W}) \in \mathbb{R}^{(d+1)N},$$

we can use the Gauss-Newton iteration (where A^+ is the Moore-Penrose pseudoinverse)

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha_k [\nabla(\boldsymbol{F}(\boldsymbol{z}^k))]^+ \boldsymbol{F}(\boldsymbol{z}^k), \tag{3.14}$$

to find roots of (3.13), where $\alpha \in (0, 1)$. As we show in Section 3.2.1, we have access to analytical formula for computing the derivatives of (3.13), which enables us to alternatively use the damped Newton iteration

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha [\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}^{\boldsymbol{k}}))]^{-1} \nabla(\boldsymbol{F}(\boldsymbol{z}^k)). \tag{3.15}$$

We also consider the Newton iteration for finding roots of

$$||\boldsymbol{F}(\boldsymbol{z})||_2^2 := \boldsymbol{f} : \mathbb{R}^{3N} \to \mathbb{R}.$$

Namely,

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha [\mathcal{D}^2(\boldsymbol{f}(\boldsymbol{z}^k))]^{-1} \nabla(\boldsymbol{f}(\boldsymbol{z}^k)) \tag{3.16}$$

The choice of initial point $\boldsymbol{z}^0 = (\boldsymbol{X}_1^0, \cdots, \boldsymbol{X}_d^0, \boldsymbol{W}_0)$ and control of the step length α_k are vital for its convergence. Moreover, we'd like to incorporate inequality constraints on $\boldsymbol{z}^k$ to enforce that quadrature nodes are interior to $\mathcal{T}^2$, and we want the weights to be positive with unit sum. By casting the problem of finding roots of (3.13)-(3.16) into a minimization problem followed by a relaxation of $\boldsymbol{F}$, we can use techniques from constrained convex optimization to find acceptable quadratures. The optimization problem we would like to solve is

$$\begin{aligned}
&\min_{\boldsymbol{z} \in \mathbb{R}^{(d+1)N}} ||\boldsymbol{F}(\boldsymbol{z})||_2^2 \quad \text{such that} \\
&G\boldsymbol{z} < g, \ \begin{bmatrix} \boldsymbol{0_{1 \times dN}} & \boldsymbol{1_{1 \times N}} \end{bmatrix} \boldsymbol{z} = 1, \quad \text{where} \\
&G = \begin{bmatrix} -I & \times & \cdots & \times \\ \times & & \ddots & \times \\ \vdots & & \ddots & \vdots \\ \times & \cdots & \times & -I \\ I & \cdots & I & \times \end{bmatrix} \in \mathbb{R}^{(d+2)N \times 3N}, \quad g = \begin{bmatrix} \boldsymbol{0_{dN \times 1}} \\ \boldsymbol{1_{2N \times 1}} \end{bmatrix} \in \mathbb{R}^{(d+3)M}.
\end{aligned} \tag{3.17}$$

The zero entries in the constraint matrix G are denoted by $\times$, and I is the $N \times N$ identity matrix. Specifically, each block row of G corresponds, respectively, to the following inequality constraints: (i) $\boldsymbol{x}_1 > 0, \cdots, \boldsymbol{x}_d, \boldsymbol{w} > 0$, (ii) $\sum_{i=1}^d \boldsymbol{x}_i < 1$, and the equality constraint is $\sum_{i=1}^N w_i = 1$. *If a local minimum $\boldsymbol{z}$ is found, it is not necessarily a zero of $\boldsymbol{F}(\boldsymbol{z})$. We have only satisfied the necessary KKT conditions. However, we may be closer to an actual root and will have maintained interior nodes and positive weights.* Hence, we can use this new estimate $\boldsymbol{z}$ as an initial point in a Newton iteration that will relax $\boldsymbol{F}(\boldsymbol{z})$ to zero quadratically. We can solve the quadratic program (3.17) with the sequential quadratic programming algorithm.

Ideally, we won't have to use a quadratic program for solving (3.17), and Newton descent will be sufficient.

3.2.1. Derivatives of the objective functions. We can use the machinery developed in Section 1.1 to compute derivatives of (3.13). Given that these tools are only at our disposal for $d \leq 2$, we proceed for now with $d = 2$ and $\boldsymbol{x} \in \mathcal{T}^2$. We view each component of the objective as a function $F_j : \mathbb{R}^{3N} \to \mathbb{R}$, and note that $\boldsymbol{P}$ is defined in the (a, b, c) Jacobi basis. Applying the derivative operators defined in (1.7), we have at a point $(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W})$,

$$
\begin{aligned}
&F_j(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W}) = \boldsymbol{e}_j^T \boldsymbol{P}(\boldsymbol{X}, \boldsymbol{Y}) \boldsymbol{W} - \delta_{1j} = \sum_{i=1}^{N} P_j(x_i, y_i) w_i - \delta_{1j}, \\
&\frac{\partial F_j}{\partial x_k} = w_k \boldsymbol{P}^x(x_k, y_k)^T \mathcal{D}_x \boldsymbol{e}_j, \quad \frac{\partial F_j}{\partial y_k} = w_k \boldsymbol{P}^y(x_k, y_k)^T \mathcal{D}_y \boldsymbol{e}_j, \quad \frac{\partial F_j}{\partial w_k} = \boldsymbol{P}(x_k, y_k)^T \boldsymbol{e}_j, \\
&\frac{\partial^2 F_j}{\partial x_l \partial x_k} = w_k \boldsymbol{P}^{xx}(x_k, y_k)^T \mathcal{D}_{xx} \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial y_k} = w_k \boldsymbol{P}^{xy}(x_k, y_k)^T \mathcal{D}_{xy} \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial w_k} = \boldsymbol{P}^x(x_k, y_k)^T \mathcal{D}_x \boldsymbol{e}_j \delta_{lk}, \\
&\frac{\partial^2 F_j}{\partial y_l \partial x_k} = w_k \boldsymbol{P}^{yx}(x_k, y_k)^T \mathcal{D}_{yx} \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial y_k} = w_k \boldsymbol{P}^{yy}(x_k, y_k)^T \mathcal{D}_{yy} \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial w_k} = \boldsymbol{P}^y(x_k, y_k)^T \mathcal{D}_y \boldsymbol{e}_j \delta_{lk}, \\
&\frac{\partial^2 F_j}{\partial w_l \partial x_k} = \boldsymbol{P}^x(x_k, y_k)^T \mathcal{D}_x \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial y_k} = \boldsymbol{P}^y(x_k, y_k)^T \mathcal{D}_y \boldsymbol{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial w_k} = 0 \cdot \delta_{lk},
\end{aligned}
$$

where

$$
\begin{aligned}
&\mathcal{D}_x = \mathcal{D}^{a+1,b,c+1}_{x,(a,b,c)}, \quad \boldsymbol{P}^x = \boldsymbol{P}^{(a+1,b,c+1)}, \quad \mathcal{D}_y = \mathcal{D}^{a,b+1,c+1}_{x,(a,b,c)}, \quad \boldsymbol{P}^y = \boldsymbol{P}^{(a,b+1,c+1)} \\
&\mathcal{D}_{xx} = \mathcal{D}^{(a+2,b,c+2)}_{x,(a+1,b,c+1)} \mathcal{D}^{(a+1,b,c+1)}_{x,(a,b,c)}, \quad \boldsymbol{P}^{xx} = \boldsymbol{P}^{(a+2,b,c+2)}, \\
&\mathcal{D}_{yy} = \mathcal{D}^{(a,b+2,c+2)}_{y,(a,b+1,c+1)} \mathcal{D}^{(a,b+1,c+1)}_{y,(a,b,c)}, \quad \boldsymbol{P}^{yy} = \boldsymbol{P}^{(a,b+2,c+2)} \\
&\mathcal{D}_{xy} = \mathcal{D}^{(a+1,b+1,c+2)}_{x,(a,b+1,c+1)} \mathcal{D}^{(a,b+1,c+1)}_{y,(a,b,c)}, \quad \boldsymbol{P}^{xy} = \boldsymbol{P}^{(a+1,b+1,c+2)}, \\
&\mathcal{D}_{yx} = \mathcal{D}^{(a+1,b+1,c+2)}_{y,(a+1,b,c+1)} \mathcal{D}^{(a+1,b,c+1)}_{x,(a,b,c)}, \quad \boldsymbol{P}^{yx} = \boldsymbol{P}^{xy}
\end{aligned}
$$

Now, reverting to the view of the objective as $\boldsymbol{F} : \mathbb{R}^{3N} \to \mathbb{R}^M$, we can write

$$
[\mathcal{D}(\boldsymbol{F})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} = \begin{bmatrix} \mathcal{D}_x^T \boldsymbol{P}^x(\boldsymbol{X}, \boldsymbol{Y}) \odot \boldsymbol{W} & \mathcal{D}_y^T \boldsymbol{P}^y(\boldsymbol{X}, \boldsymbol{Y}) \odot \boldsymbol{W} & \boldsymbol{P}(\boldsymbol{X}, \boldsymbol{Y}) \end{bmatrix} \in \mathbb{R}^{M \times 3N}, \tag{3.18}
$$

and we can use this formula in the Gauss-Newton update (3.14). If we instead consider the equivalent problem of finding zeros of the objective norm $\boldsymbol{f} : \mathbb{R}^{3N} \to \mathbb{R}$, $\boldsymbol{f}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W}) = ||\boldsymbol{F}||_2^2$, (solved by the iteration (3.16)), we must chain out derivatives to find the mapping $\mathcal{D}(\boldsymbol{f}) : \mathbb{R}^{3N} \to L(\mathbb{R}^{3N}, \mathbb{R})$ at a point $(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W})$ is given by

$$
[\mathcal{D}(\boldsymbol{f})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} = 2\boldsymbol{F}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W})^T [\mathcal{D}(\boldsymbol{F})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} \in \mathbb{R}^{1 \times 3N}, \tag{3.19}
$$

where $L(U, V)$ is the space of linear mappings between finite dimensional vector spaces U, V. In this case, the Hessian matrix of $\boldsymbol{f}$ is a mapping $\mathcal{D}^2(\boldsymbol{f}) : \mathbb{R}^{3N} \to L(\mathbb{R}^{3N}, \mathbb{R}^{3N})$ given at a point $(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W})$ by

$$
\begin{aligned}
[\mathcal{D}^2(\boldsymbol{f})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} = &\, 2 \sum_{j=1}^{M} F_j(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{W}) [\mathcal{D}^2(F_j)]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} \\
&+ 2 \Big([\mathcal{D}(\boldsymbol{F})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} \Big)^T [\mathcal{D}(\boldsymbol{F})]|_{(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})} \\
&\in \mathbb{R}^{3N \times 3N}
\end{aligned} \tag{3.20}
$$

The vector Hessian of $\boldsymbol{F}$ appears in (3.20), and we can make sense of its tensor structure by reviewing the mixed partial formulae given at the beginning of this section. In particular, we have

$$\partial_{\boldsymbol{XX}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{W}\odot\boldsymbol{P}^{xx}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{xx}\boldsymbol{e}_j\right)\right), \tag{3.21}$$

$$\partial_{\boldsymbol{YX}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{W}\odot\boldsymbol{P}^{yx}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{yx}\boldsymbol{e}_j\right)\right), \tag{3.22}$$

$$\partial_{\boldsymbol{WX}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{P}^{x}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{x}\boldsymbol{e}_j\right)\right), \tag{3.23}$$

$$\partial_{\boldsymbol{XY}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{W}\odot\boldsymbol{P}^{xy}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{xy}\boldsymbol{e}_j\right)\right), \tag{3.24}$$

$$\partial_{\boldsymbol{YY}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{W}\odot\boldsymbol{P}^{yy}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{yy}\boldsymbol{e}_j\right)\right), \tag{3.25}$$

$$\partial_{\boldsymbol{WY}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{P}^{y}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{y}\boldsymbol{e}_j\right)\right), \tag{3.26}$$

$$\partial_{\boldsymbol{XW}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{P}^{x}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{x}\boldsymbol{e}_j\right)\right), \tag{3.27}$$

$$\partial_{\boldsymbol{YW}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = I_{N\times N}\odot\left(\mathbf{1}_N^T\otimes\left(\boldsymbol{P}^{y}(\boldsymbol{X},\boldsymbol{Y})^T\mathcal{D}_{y}\boldsymbol{e}_j\right)\right), \tag{3.28}$$

$$\partial_{\boldsymbol{WW}}(\boldsymbol{F}_j(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{W})) = 0_{N\times N}, \tag{3.29}$$

where $\cdot\otimes\cdot$ symbolizes the Kronecker product, and $\cdot\odot\cdot$ is component-wise multiplication along the columns of the argument to the right[2]. We verify the correctness of our formulae for the first and second derivatives of our vector and scalar objectives by comparing with second-order finite difference approximations. In particular, we show in Figure 2 the relative error in the 2-norm error between the analytical evaluation and the finite difference approximation. Since the difference scheme is second order, we expect the error to behave like $\mathcal{O}(h^2)$, where h is the stencil width. That is, we should observe that

$$\log(||\mathcal{D}(\boldsymbol{F}) - \hat{\mathcal{D}}(\boldsymbol{F})||) \approx 2\log(h) + \beta,$$

where β is the order constant, and $\hat{\mathcal{D}(F)}$ is the finite difference approximation. The same relationship holds for $\mathcal{D}^2(F)$ and other derivatives as well. On a $\log - \log$ scale, this means the slope of the error lines should be around 2. Indeed, we do observe this relationship in the regime of asymptotic convergence, before round-off error dominates truncation error for the finite difference scheme.

[2]Viewing $\boldsymbol{W}$ as a column of size N, we compute the component-wise product with columns of the matrix on the right. It would be more correct to define a function `repmat`(W,k), which takes a vector $W\in\mathbb{R}^N$ and repeats it k times, producing a matrix in $\mathbb{R}^{N\times k}$ with each of the k columns a copy of W, but for $N\neq M$, the ambiguity in matching sizes of the arguments to $\odot$ is eliminated.

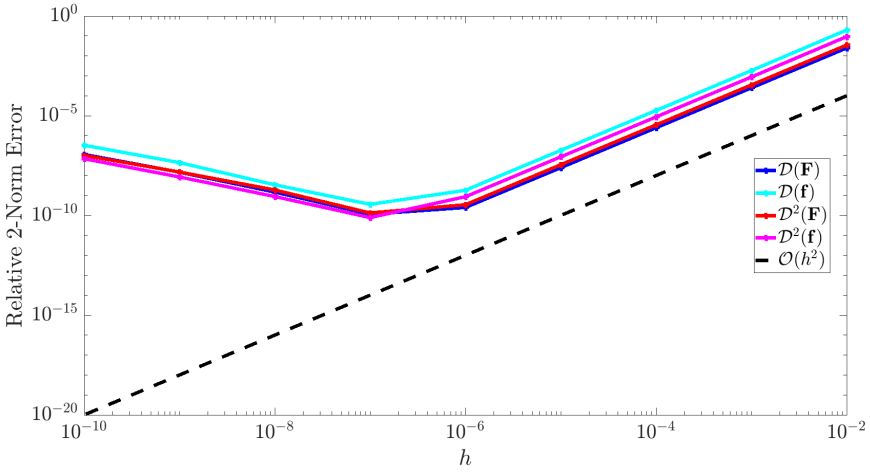

FIG. 2. *Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae when compared to second order finite difference approximations. We see that our formulae must be correct, if our finite difference implementation is correct (which is an initial assumption). The kink in the error curves after which the error increases is typical of a finite difference scheme, and occurs at the "usual" locaton for at least twice differentiable functons, i.e.* $h \approx 1e-6$. *This corresponds to the point where numerical round-off error (due to IEEE floating point specification) dominates truncation error, and can be explicitly determined via error analysis by making reference to the Taylor error term in the difference scheme, with propagation of* ε_{mach}

The plot also illustrates the utility of having available to us the analytical derivative formulae. In particular, we note that the solution variables to any of the proposed iterations involve nodes interior to $\mathcal{T}^2$, and the zero of the objective function is only some ε perturbation away from the initial points obtained by `JEVD` or complex eigenvalue decomposition. If this ε is smaller than 1e-6, which I've empirically observed is the case, finite difference schemes are not appropriate for approximating the vector Hessian or gradient of the objective.

Remark 3.1. For this sort of simple "reverse" verification test, we assume that we don't know if our analytical formulae are correct, and that a *verified* finite difference implementation is available. Then, we are essentially re-verifying the finite difference implementation, which on success, implies correctness of the analytical formulae.

At this point, we can parse out what we mean by applying the inverse of the vector Hessian of $\boldsymbol{F} : \mathbb{R}^{3N} \to \mathbb{R}^M$ to the vector gradient in (3.15). We know that

$$\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z})) \in \mathbb{R}^{M\times 3N\times 3N}, \quad \mathcal{D}(\boldsymbol{F}(\boldsymbol{z})) \in \mathbb{R}^{M\times 3N}.$$

Then, we take the following natural interpretation[3]

$$[\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}))]^{-1} := \begin{bmatrix}[\mathcal{D}^2(F_1(\boldsymbol{z}))]^{-1} & \cdots & [\mathcal{D}^2(F_M(\boldsymbol{z}))]^{-1}\end{bmatrix}$$

$$\nabla(\boldsymbol{F}(\boldsymbol{z})) := \begin{bmatrix}\mathcal{D}(F_1(\boldsymbol{z}))^T \\ \vdots \\ \mathcal{D}(F_M(\boldsymbol{z}))^T\end{bmatrix}$$

$$\Rightarrow [\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}))]^{-1}\mathcal{D}(\boldsymbol{F}(\boldsymbol{z})) := \sum_{j=1}^{M} \mathcal{D}^2(F_j(\boldsymbol{z}))^{-1}\nabla(F_j(\boldsymbol{z})) \in \mathbb{R}^{3N\times 1}, \quad \text{(as required)}$$

[3]For a function $\boldsymbol{u} : \mathbb{R}^N \to \mathbb{R}^M$, we note the distinction between $\mathcal{D}(\boldsymbol{u}(\boldsymbol{z}))$ and $\nabla(\boldsymbol{u}(\boldsymbol{z}))$. The former is an element of $L(\mathbb{R}^N, \mathbb{R}^M)$, equivalently viewed as a set of M elements of $L(\mathbb{R}^N, \mathbb{R})$, or M row vectors in $\mathbb{R}^N$ (left argument of dot product). The latter evaluated at a point is as an element of $L(\mathbb{R}^M, \mathbb{R}^N)$, equivalently viewed as a set of M elements of $L(\mathbb{R}, \mathbb{R}^N)$, or M column vectors in $\mathbb{R}^N$ (right argument of dot product). TLDR; $[\mathcal{D}(\boldsymbol{u})]^T = \nabla(\boldsymbol{u})$